TODO:

- Ausfüllrätsel
- Entscheidbarkeit
- Halteproblem
- Liste der NP-Vollständigen Probleme
- TM Berechnungsgeschichte
- extras
  - CNF konstruieren
  - Minimalautomat
  - Mengenoperationen
  - DEA/NEA/TM/NTM
- Beispiele:
  - Ist Sprache x regulär? → Pumping Lemma
  - Ist Sprache x kontextfrei? → Pumping Lemma
  - Kann eine NTM in polynomieller Zeit entscheiden, ob ein Ausfüllrätsel eine Lösung hat? → Satz von Rice / P Verifizierer / Reduktion
  - Ist Grammatik eindeutig? (FS_2026)5))
  - Parse-tree finden (FS_2022)5))
- Prüfungsaufgaben:
  - HS_2021)4)
  - FS_2022)4)

Begriff	Bedeutung
Regulär	Es gibt einen DEA A, der L akzeptiert, also $L(A) = L$
Kontextfrei	L kann nur von einem ND PDA erkannt werden

PUMPING LEMMA (DEA)

Um zu beweisen, dass eine Sprache **nicht** regulär ist

Vorgehen:
1) Annahme: L ist regulär
2) Gemäss Pumping Lemma gibt es die Pumping Length N
3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
4) Aufteilung des Wortes gemäss Pumping Lemma
   - $w = xyz, |xy| \leq N, |y| > 0$
5) Auswirkung des Pumpens
   - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
6) Widerspruch ⇒ Annahme kann nicht zutreffen

TODO:
Beispiel

PUMPING LEMMA (CFL)

Um zu beweisen, dass eine Sprache **nicht** kontextfrei ist

Voraussetzungen:
1) $|vy| > 0$
2) $|vxy| \leq N$
3) $uv^kxy^kz \in L \forall k \in \mathbb{N}$

TODO:
Beispiel

CHOMSKY-NORMALFORM (CNF)

- S kommt auf der rechten Seite nicht vor
- Jede Regel von der Form $A \to BC$ oder $A \to a$
- $S \to \varepsilon$ ist erlaubt

ENTSCHEIDBARKEIT

SATZ VON RICE

Ist P eine nichttriviale Eigenschaft Turing-erkennbarer Sprachen, dann ist P_{TM} nicht entscheidbar.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst ***nichttrivial***, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

LÖSUNGSANLEITUNG EINER PRÜFUNGSFRAGE:

- Nichttriviale Eigenschaft P aufschreiben
- Die beiden Sprachen $L(M_1)$ und $L(M_2)$ bilden (meistens kann man die leere Menge oder Σ^* als eine dieser Sprachen verwenden)
- Gibt es ein Programm, welches beide Sprachen erkennen kann? Sind beide Sprachen Turing erkennbar?
- Dann besagt der Satz von Rice, dass die Sprache nicht entscheidbar ist

TODO:
Beispiel

NP

TODO:

POLYNOMIELLER VERIFIZIERER

TODO:

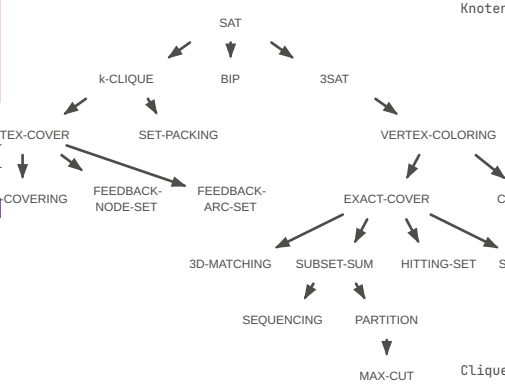
REDUKTION

TODO:

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

TODO:




```
(
  sequence(
    [ ],
    heading(depth: 4, body: [Graph-Überdeckungen]),
    parbreak(),
    sequence(
      [ ],
        [Handeln davon, dass ein gegebener Graph von einer Menge beschränkter],
      [ ],
        [Größe mit einer besonderen Eigenschaft aus erreicht werden kann.],
      [ ],
    ),
    parbreak(),
    sequence(
      [ ],
      heading(depth: 5, body: [VERTEX-COLORING]),
      parbreak(),
      sequence(
        [ ],
        [Gegeben ist ein Graph],
        [ ],
        equation(
          block: false,
          body: sequence(
            [G],
            [ ],
            [=],
            [ ],
              lr(body: sequence([([], [V], [,], [ ], [E], [])])),
          ),
        ),
        [ ],
        [mit Knoten],
        [ ],
        equation(block: false, body: [V]),
        [ ],
        [und Kanten],
        [ ],
        equation(block: false, body: [K]),
        [ ],
        [und],
        [ ],
        [eine Zahl],
        [ ],
        equation(block: false, body: [k]),
       [. Ist es möglich, die Knoten des Graphen mit],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [Farben],
        [ ],
        [so einzufärben, dass durch Kanten verbundene Knoten verschiedene],
        [ ],
        [Farben haben?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [CLIQUE-COVER]),
      parbreak(),
      sequence(
        [ ],
        [Gibt es],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [Cliquen so, dass jede Ecke in genau einer der Cliquen ist?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [EXACT-CLIQUE-COVER]),
      parbreak(),
      sequence(
        [ ],
          [Wie CLIQUE-COVER, nur müssen die Mengen disjunkt sein.],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [K-CLIQUE]),
      parbreak(),
      sequence(
        [ ],
        [Wie CLIQUE-COVER, nur ist],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [bekannt.],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [VERTEX-COVER]),
      parbreak(),
      sequence(
        [Gibt es eine Menge],
        [ ],
        equation(block: false, body: [W]),
        [ ],
        [von],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [Knoten derart, dass jede Kante von],
        [ ],
        equation(block: false, body: [G]),
        [ ],
        [einen Endpunkt in],
        [ ],
        equation(block: false, body: [W]),
        [ ],
        [hat?],
      ),
      [ ],
      [ ],
      [ ],
    ),
    [ ],
  ),
  sequence(
    [ ],
    heading(depth: 4, body: [Mengen]),
    parbreak(),
    sequence(
      [ ],
      [Handeln von einer Familie],
      [ ],
      equation(
        block: false,
        body: attach(
          base: lr(
            body: sequence([([], attach(base: [S], b: [i]), [])]),
          ),
          b: sequence([i], [ ], [∈], [ ], [I]),
        ),
      ),
      [ ],
      [von nichtleeren endlichen],
      [ ],
        [Mengen. In diesen Problemen geht es um die existenz einer Teilfamilie],
      [ ],
      [mit bestimmten Eigenschaften.],
      [ ],
    ),
    parbreak(),
    sequence(
      [ ],
      heading(depth: 5, body: [SET-PACKING]),
      parbreak(),
      sequence(
        [ ],
        [Wie viele der Mengen],
        [ ],
       equation(block: false, body: attach(base: [S], b: [i])),
        [ ],
        [kann man in],
        [ ],
        equation(block: false, body: [U]),
        [ ],
        [hineinpacken, ohne dass sie],
        [ ],
        [sich überlappen? Gibt es],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [Teilmengen in],
        [ ],
        equation(block: false, body: [S]),
        [, welche disjunkt sind],
        [ ],
        [und die Menge],
        [ ],
        equation(block: false, body: [U]),
        [ ],
        [abdecken?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [SET-COVERING]),
      parbreak(),
      sequence(
        [ ],
        [Gibt es eine Unterfamilie bestehend aus],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [Mengen, die die gleiche],
        [ ],
        [Vereinigung],
        [ ],
        equation(block: false, body: [U]),
        [ ],
        [hat? Kann man],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [Teilmengen bilden, welche die Menge],
        [ ],
        equation(block: false, body: [S]),
        [ ],
        [komplett abdecken?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [EXACT-COVER]),
      parbreak(),
      sequence(
        [ ],
          [Gibt es eine Unterfamilie von Mengen, die disjunkt sind, aber die],
        [ ],
        [gleiche Vereinigung haben? Jedes Element in],
        [ ],
        equation(block: false, body: [U]),
        [ ],
        [soll genau in einer],
        [ ],
        [der Teilmengen der Familie],
        [ ],
        equation(block: false, body: [S]),
        [ ],
        [vorkommen. Die gesuchte Menge bildet],
        [ ],
        [eine exakte Überdeckung.],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [HITTING-SET]),
      parbreak(),
      sequence(
        [ ],
        [Sucht zu],
        [ ],
        equation(block: false, body: [U]),
        [ ],
        [eine Menge von Punkten],
```

```
        [ ],
        [(Elemente), die jede Menge der],
        [ ],
        [Familie in genau einem Punkt treffen. Gibt es eine Menge],
        [ ],
        equation(
          block: false,
          body: sequence([W], [ ], [⊂], [ ], [U]),
        ),
        [ ],
        [derart, dass],
        [ ],
        equation(block: false, body: [W]),
        [ ],
        [mit jeder Menge],
        [ ],
        equation(block: false, body: attach(base: [S], b: [i])),
        [ ],
        [nur ein Element gemeinsam hat?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
    ),
    [ ],
  ),
  sequence(
    [ ],
      heading(depth: 4, body: [Aufteilung in zwei Mengen]),
    parbreak(),
    [],
    parbreak(),
    sequence(
      [ ],
      heading(depth: 5, body: [PARTITION]),
      parbreak(),
      sequence(
        [ ],
        [Gibt es eine Unterteilung einer Liste],
        [ ],
        equation(block: false, body: [S]),
        [ ],
        [von ganzen Zahlen in zwei],
        [ ],
        [disjunkte Listen mit gleicher Summe?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [MAX-CUT]),
      parbreak(),
      sequence(
        [ ],
        [Gegeben ein Graph],
        [ ],
        equation(
          block: false,
          body: sequence(
            [G],
            [ ],
            [=],
            [ ],
              lr(body: sequence([(], [V], [,], [ ], [E], [)])),
          ),
        ),
        [ ],
        [mit einer Gewichtsfunktion],
        [ ],
        equation(
          block: false,
          body: sequence([φ], [:], [ ], [E], [ ], [→], [ ], [ℤ]),
        ),
        [ ],
        [der Kanten und einer ganzen Zahl],
        [ ],
        equation(
          block: false,
          body: sequence([w], [ ], [∈], [ ], [ℤ]),
        ),
        [, gibt es],
        [ ],
        [eine Aufteilung der Knotenmenge],
        [ ],
        equation(
          block: false,
          body: sequence(
            [V],
            [ ],
            [=],
            [ ],
            attach(base: [V], b: [1]),
            [ ],
            [U],
            [ ],
            attach(base: [V], b: [2]),
          ),
        ),
        [ ],
        [in zwei disjunkte],
        [ ],
      [Teilmengen, so dass das Gewicht der Kanten, die],
        [ ],
        equation(block: false, body: attach(base: [V], b: [1])),
        [ ],
        [und],
        [ ],
        equation(block: false, body: attach(base: [V], b: [2])),
        [ ],
        [verbinden, mindestens],
        [ ],
        equation(block: false, body: [w]),
        [ ],
        [ist],
        [:],
        [ ],
        equation(
          block: true,
          body: sequence(
            sequence(
              [φ],
              lr(
                body: sequence(
                  [(],
                  attach(base: [V], b: [1]),
                  [,],
                  attach(base: [V], b: [2]),
                  [)],
                ),
              ),
            ),
            [ ],
            [=],
            [ ],
            attach(
              base: [∑],
              b: sequence(
                attach(base: [v], b: [1]),
                [ ],
                [∈],
                [ ],
                attach(base: [v], b: [2]),
                [ ],
                [∈],
                [ ],
                attach(base: [V], b: [1]),
                [,],
                [ ],
                attach(base: [v], b: [2]),
                [ ],
                [∈],
                [ ],
                [E],
              ),
            ),
            [ ],
              sequence([φ], lr(body: sequence([(], [e], [)]))),
            [ ],
            [≥],
            [ ],
            [w],
            [?],
          ),
        ),
        [ ],
      ),
      [ ],
      [ ],
      [ ],
    ),
    [ ],
  ),
  sequence(
    [ ],
        heading(depth: 4, body: [Kombinatirische Optimierung]),
    parbreak(),
    sequence(
      [ ],
    [Handelt um das finden eines Maximums oder Minimums einer Zielfunktion.],
      [ ],
    ),
    parbreak(),
    sequence(
      [ ],
      heading(depth: 5, body: [STEINER-TREE]),
      parbreak(),
      sequence(
        [ ],
        [Gibt es zu einem Graphen],
        [ ],
        equation(
          block: false,
          body: sequence(
            [G],
            [ ],
            [=],
            [ ],
              lr(body: sequence([(], [V], [,], [ ], [E], [)])),
          ),
        ),
        [ ],
        [mit einer Gewichtsfunktion],
        [ ],
        equation(
          block: false,
          body: sequence([w], [:], [ ], [E], [ ], [→], [ ], [ℤ]),
        ),
        [, einer Teilmenge],
        [ ],
        equation(
          block: false,
          body: sequence([R], [ ], [⊂], [ ], [V]),
        ),
        [ ],
        [der Knoten und einer Zahl],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [einen Teilbaum],
        [ ],
        equation(
          block: false,
          body: sequence(
            [T],
            [ ],
            [=],
            [ ],
            lr(
              body: sequence(
                [(],
                attach(base: [V], b: [1]),
                [,],
                [ ],
                attach(base: [E], b: [1]),
                [)],
              ),
            ),
            [ ],
            [⊂],
            [ ],
            [G],
          ),
        ),
        [ ],
        [des Graphen, der alle],
        [ ],
        [Knoten von],
        [ ],
        equation(block: false, body: [R]),
        [ ],
        [enthält und dessen Kantengewicht],
        [ ],
        equation(
          block: false,
          body: sequence(
            attach(
              base: [∑],
              b: sequence([e], [ ], [∈], [ ], attach(base: [E], b: [1])),
            ),
            [ ],
              sequence([w], lr(body: sequence([(], [e], [)]))),
            [ ],
            [≤],
            [ ],
            [k],
          ),
        ),
        [ ],
        [ist?],
        [ ],
      ),
      [ ],
    ),
    [ ],
    [ ],
    [ ],
    [ ],
    heading(depth: 5, body: [SEQUENCING]),
    parbreak(),
    sequence(
      [ ],
      [Die Komponenten der drei Vektoren],
      [ ],
      equation(
        block: true,
        body: sequence(
          [Laufzeiten:],
          [ ],
          align-point(),
          [ ],
          [T],
          [ ],
          [=],
          [ ],
          align-point(),
          align-point(),
          [ ],
          lr(
            body: sequence(
              [(],
              attach(base: [T], b: [1]),
              [ ],
              [,],
              [ ],
              […],
              [ ],
              [,],
              [ ],
              attach(base: [T], b: [p]),
              [)],
            ),
          ),
          [ ],
          align-point(),
          align-point(),
          [ ],
          [∈],
          [ ],
          attach(base: [ℤ], t: [p]),
          [ ],
          linebreak(),
          [ ],
          [Deadlines:],
          [ ],
          align-point(),
          [ ],
          [D],
          [ ],
          [=],
          [ ],
          align-point(),
          align-point(),
          [ ],
          lr(
            body: sequence(
              [(],
              attach(base: [D], b: [1]),
              [ ],
              [,],
              [ ],
              […],
              [ ],
              [,],
              [ ],
              attach(base: [D], b: [p]),
              [)],
```

```
            ),
          ),
          [ ],
          align-point(),
          align-point(),
          [ ],
          [∈],
          [ ],
          attach(base: [ℤ], t: [p]),
          [ ],
          linebreak(),
          [ ],
          [Strafen:],
          [ ],
          align-point(),
          [ ],
          [P],
          [ ],
          [=],
          [ ],
          align-point(),
          align-point(),
          [ ],
          lr(
            body: sequence(
              [(],
              attach(base: [P], b: [1]),
              [ ],
              [,],
              [ ],
              […],
              [ ],
              [,],
              [ ],
              attach(base: [P], b: [p]),
              [)],
            ),
          ),
          [ ],
          align-point(),
          align-point(),
          [ ],
          [∈],
          [ ],
          attach(base: [ℤ], t: [p]),
        ),
      ),
      [ ],
      [beschreiben je einen Job. Job],
      [ ],
      equation(block: false, body: [i]),
      [ ],
      [hat Laufzeit],
      [ ],
      equation(block: false, body: attach(base: [T], b: [i])),
      [, sollte zur Zeit],
      [ ],
      equation(block: false, body: attach(base: [D], b: [i])),
      [ ],
      [abgeschlossen sein und kostet die Strafe],
      [ ],
      equation(block: false, body: attach(base: [P], b: [i])),
      [, wenn er nicht],
      [ ],
        [rechtzeitig fertig ist. Die Jobs werden sequenziell ohne unterbruch],
      [ ],
      [abgearbeitet. Ist es möglich, die Reihenfolge der Ausführung der Jobs],
      [ ],
      [so zu planen, dass die entstehenden Strafen],
      [ ],
          equation(block: false, body: sequence([≤], [ ], [k])),
          [ ],
          [sind?],
          [ ],
        ),
        [ ],
        [ ],
        [ ],
      ),
      [ ],
    ),
    sequence(
      [ ],
      heading(depth: 4, body: [Pfade in Graphen]),
      parbreak(),
      sequence(
        [ ],
        [Ein hamiltonscher Pfad in einem Graphen],
        [ ],
        equation(
          block: false,
          body: sequence(
            [G],
            [ ],
            [=],
            [ ],
              lr(body: sequence([(], [V], [,], [ ], [E], [)])),
          ),
        ),
        [ ],
        [ist ein Pfad, der],
        [ ],
        [jeden Knoten des Graphen genau einmal besucht. Ein solcher Pfad],
        [ ],
        [definiert eine Reihenfolge],
        [ ],
        equation(
          block: false,
          body: sequence(
            attach(base: [a], b: [1]),
            [,],
            [ ],
            […],
            [ ],
            [,],
            [ ],
            attach(base: [a], b: [n]),
          ),
        ),
        [ ],
        [von Knoten, so dass jede],
        [ ],
        [Kante von],
        [ ],
        equation(block: false, body: attach(base: [a], b: [i])),
        [ ],
        [nach],
        [ ],
        equation(
          block: false,
            body: attach(base: [a], b: sequence([i], [+], [1])),
        ),
        [ ],
        [in],
        [ ],
        equation(block: false, body: [E]),
        [ ],
        [ist. Für einen gerichteten Graphen],
        [ ],
        [sind dies die Kanten],
        [ ],
        equation(
          block: false,
          body: sequence(
            [(],
            attach(base: [a], b: [i]),
            [,],
            [ ],
                attach(base: [a], b: sequence([i], [+], [1])),
            [)],
          ),
        ),
        [ ],
        [∈],
        [ ],
        [E],
      ),
      [, für einen ungerichteten],
      [ ],
      [Graphen gilt],
      [ ],
      equation(
        block: false,
        body: sequence(
          lr(
            body: sequence(
              [{],
              attach(base: [a], b: [i]),
              [,],
              [ ],
                  attach(base: [a], b: sequence([i], [+], [1])),
              [}],
            ),
          ),
          [ ],
          [∈],
          [ ],
          [E],
        ),
      ),
      [. Ein hamiltonscher Zyklus ist ein],
      [ ],
      [geschlossener hamiltonscher Pfad.],
      [ ],
    ),
    parbreak(),
    sequence(
      [ ],
      heading(depth: 5, body: [HAMPATH]),
      parbreak(),
      sequence(
        [ ],
        [Gibt es einen gerichteten hamiltonschen Pfad im gerichteten Graphen],
        [ ],
        equation(block: false, body: [G]),
        [?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [HAMCIRCUIT]),
      parbreak(),
        [Wie HAMPATH, aber zusätzlich muss der Pfad geschlossen sein],
      [ ],
      [ ],
      [ ],
    ),
    sequence(
      [ ],
      heading(depth: 5, body: [UHAMPATH]),
      parbreak(),
      [Wie HAMPATH, aber ungerichtet],
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [UHAMCIRCUIT]),
      parbreak(),
        [Wie UHAMPATH, aber zusätzlich muss der Pfad geschlossen sein],
      [ ],
      [ ],
      [ ],
    ),
    [ ],
  ),
  sequence(
    [ ],
    heading(depth: 4, body: [Gerichtete Graphen]),
    parbreak(),
    sequence(
      [ ],
      [Es handelt sich um die Eigenschaften von Zyklen in gerichteten Graphen],
      [ ],
      equation(
        block: false,
        body: sequence(
          [G],
          [ ],
          [=],
          [ ],
          lr(body: sequence([(], [V], [,], [E], [)])),
        ),
      ),
      [. Die Kantenmenge],
      [ ],
      equation(block: false, body: [E]),
      [ ],
      [besteht aus gerichteten Kanten],
      [ ],
      equation(
        block: false,
        body: lr(body: sequence([(], [a], [,], [ ], [e], [ ], [∈], [ ], [V])),
      ),
      [ ],
      [mit],
      [ ],
      equation(
        block: false,
        body: sequence([a], [,], [ ], [e], [ ], [∈], [ ], [V]),
      ),
      [. Ein gerichteter Zyklus in],
      [ ],
      equation(block: false, body: [G]),
      [ ],
      [ist eine Folge von Knoten],
      [ ],
      equation(
        block: false,
        body: sequence(
          attach(base: [a], b: [0]),
          [,],
          [ ],
          […],
          [ ],
          [,],
          [ ],
          attach(base: [a], b: [n]),
          [ ],
          [=],
          [ ],
          attach(base: [a], b: [0]),
        ),
      ),
      [, die jeweils Enden von Kanten sind, also],
      [ ],
      equation(
        block: false,
        body: sequence(
          lr(
            body: sequence(
              [(],
              attach(base: [a], b: [i]),
              [,],
              [ ],
                  attach(base: [a], b: sequence([i], [+], [1])),
              [)],
            ),
          ),
          [ ],
          [∈],
          [ ],
          [E],
          [,],
          [ ],
          [i],
          [ ],
          [=],
          [ ],
          [0],
          [,],
          [ ],
          […],
          [ ],
          [,],
          [ ],
          [n],
          [-],
          [1],
        ),
      ),
      [ ],
    ),
    parbreak(),
    sequence(
      [ ],
      heading(depth: 5, body: [FEEDBACK-NODE-SET]),
      parbreak(),
      sequence(
        [ ],
        [Gibt es zu einem gerichteten Graphen],
        [ ],
        equation(block: false, body: [G]),
        [ ],
        [und einer Zahl],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [eine Menge],
        [ ],
        equation(
          block: false,
          body: sequence([W], [ ], [⊂], [ ], [V]),
        ),
        [ ],
        [von],
        [ ],
        equation(block: false, body: [k]),
        [ ],
      [Knoten derart, dass jeder geschlossene Zyklus],
        [ ],
        [durch einen dieser Knoten geht?],
        [ ],
      ),
```

```
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [FEEDBACK-ARC-SET]),
      parbreak(),
      sequence(
        [ ],
        [Gibt es zu einem gerichteten Graphen],
        [ ],
        equation(block: false, body: [G]),
        [ ],
        [und einer Zahl],
        [ ],
        equation(block: false, body: [k]),
        [ ],
        [eine Menge],
        [ ],
        equation(
          block: false,
          body: sequence([F], [ ], [⊂], [ ], [E]),
        ),
        [ ],
        [von],
        [ ],
        equation(block: false, body: [k]),
        [ ],
      [Kanten derart, dass jeder geschlossene Zyklus],
        [ ],
        [eine dieser Kanten enthält?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
    ),
    [ ],
  ),
  sequence(
    [ ],
    heading(depth: 4, body: [Weitere]),
    parbreak(),
    [],
    parbreak(),
    sequence(
      [ ],
      heading(depth: 5, body: [SAT]),
      parbreak(),
      sequence(
        [Gegeben einer aussagenlogischen Formel, gibt
es eine],
        [ ],
          [Zusammensetzung der Variablenwerte, die
die Aussage],
        [ ],
        smartquote(double: true),
        [Wahr],
        smartquote(double: true),
        [ ],
        [werden],
        [ ],
        [lassen?],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [3SAT]),
      parbreak(),
      sequence(
        [Wie SAT, nur hat jede Klausel der Normalform
maximal 3],
        [ ],
        [Literale.],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [SUBSET-SUM]),
      parbreak(),
      sequence(
        [ ],
        [Gegeben einer Liste],
        [ ],
        equation(block: false, body: [S]),
        [ ],
        [von natürlichen Zahlen],
        [ ],
        equation(
          block: false,
            body: sequence(attach(base: [s], b: [i]),
[ ], [∈], [ ], [ℕ]),
        ),
        [ ],
        [und einer],
        [ ],
        [natürlichen Zahl],
        [ ],
        equation(
          block: false,
          body: sequence([t], [ ], [∈], [ ], [ℕ]),
        ),
        [ ],
        [(Rucksack), ist es möglich eine teilliste],
        [ ],
        equation(
          block: false,
          body: sequence([T], [ ], [⊂], [ ], [S]),
        ),
        [ ],
        [auszuwählen, sodass die Elemente von],
        [ ],
        equation(block: false, body: [T]),
        [ ],
        [dies Summe],
        [ ],
        equation(
          block: false,
          body: sequence(
            [t],
            [ ],
            [=],
            [ ],
            attach(
              base: [∑],
              b: sequence([s], [ ], [∈], [ ], [T]),
            ),
            [ ],
            [s],
          ),
        ),
        [ ],
        [haben?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [3D-MATCHING]),
      parbreak(),
      sequence(
        [ ],
        [Gegeben sind drei endliche Mengen],
        [ ],
        equation(block: false, body: [X]),
        [,],
        [ ],
        equation(block: false, body: [Y]),
        [ ],
        [und],
        [ ],
        equation(block: false, body: [Z]),
        [ ],
        [mit gleich vielen],
        [ ],
        equation(
          block: false,
          body: sequence(
            [n],
            [ ],
            [=],
            [ ],
            lr(body: sequence([|], [X], [|])),
            [ ],
            [=],
            [ ],
            lr(body: sequence([|], [Y], [|])),
            [ ],
            [=],
            [ ],
            lr(body: sequence([|], [Z], [|])),
          ),
        ),
        [ ],
        [Elementen und],
        [ ],
        equation(
          block: false,
          body: sequence(
            [T],
            [ ],
            [⊂],
            [ ],
            [X],
            [ ],
            [×],
            [ ],
            [Y],
            [ ],
            [×],
            [ ],
            [Z],
          ),
        ),
        [. Gibt es eine],
        [ ],
        equation(block: false, body: [n]),
        [-elementige Teilmenge],
        [ ],
        equation(
          block: false,
          body: sequence([M], [ ], [⊂], [ ], [T]),
        ),
        [ ],
        [derart,],
        [ ],
        [dass keine zwei Tripel von],
        [ ],
        equation(block: false, body: [M]),
        [ ],
        [in einer Koordinate übereinstimmen?],
        [ ],
      ),
      [ ],
      [ ],
      [ ],
      [ ],
      heading(depth: 5, body: [BIP]),
      parbreak(),
      [Binary Integer Programming],
      [ ],
      [ ],
      [ ],
    ),
    [ ],
  ),
).join()
```